

Agent Context Compiler: 面向工具调用智能体的上下文编译器设计草案

更新时间: 2026-05-30

1. 一句话概括

Agent Context Compiler 的目标是:

把原始、混乱、动态增长的 Agent 上下文, 编译成稳定、分层、可压缩、可恢复、并且更适合 prompt cache / p

它不是 KV cache manager, 也不是普通 prompt compression。它位于:

Agent framework 之后, LLM serving backend 之前。

普通 Agent 框架通常直接拼接:

system prompt + tool schemas + memory + history + observations + current query

Agent Context Compiler 则先将这些内容解析成结构化片段, 判断每个片段的稳定性、复用范围、压缩风险和质量约束, 再生成一个更 cache-friendly 的 prompt。

2. 为什么需要上下文编译器

Tool-using Agent 的上下文有几个典型问题:

1. system prompt 和 tool schemas 很稳定, 但经常被动态字段污染, 无法形成稳定 prefix。
2. timestamp、session id、retry count、request id 等高熵字段常常被放在 prompt 开头, 直接破坏 prefix cache。
3. observation 会随工具调用快速膨胀, 导致 prefill cost 和 TTFT 增长。
4. tool schema 顺序、JSON 序列化格式、history 模板不稳定, 会让语义相同的上下文在文本层面不一致。
5. 普通 truncation 或 summarization 虽然能减少 token, 但可能删除工具调用所需的关键字段, 导致任务质量下降。

核心观点：

Cacheability is not only a serving-system property, but also an agent-design property.

中文：

上下文能不能被缓存，不只取决于 vLLM/SGLang/OpenAI API 如何管理 cache，也取决于 Agent 一开始如何构造。

3. 系统定位

Agent Context Compiler 的输入：

```
agent instruction
developer/system prompt
tool schemas
long-term memory
short-term memory
conversation history
tool call history
tool observations
current user query
runtime metadata
quality constraints
serving hints
```

Agent Context Compiler 的输出：

```
compiled prompt
segment metadata
cache keys
reuse scopes
compression decisions
recoverable references
quality guard report
```

它可以接在不同 Agent 框架后面：

LangChain / AutoGen / CrewAI / SWE-Agent / 自研 Agent framework

也可以接不同 serving backend：

vLLM / SGLang / OpenAI-compatible API / Anthropic / Gemini

4. 核心抽象：Context Segment

编译器不把 prompt 当成一整段字符串，而是把它拆成多个 segment。

推荐 segment schema：

```
{
  "name": "tool_schema_block",
  "text": "...",
  "segment_type": "tool_schema",
  "stability": "static",
  "reuse_scope": "agent_type",
  "cache_key": "sha256:...",
  "must_preserve": ["tool_name", "argument_schema", "required_fields"],
  "source_pointer": null
}
```

关键字段解释：

字段	含义
name	segment 名称，例如 system、tools、observation_03
text	segment 原始文本或压缩后文本
segment_type	system、tool_schema、memory、history、observation、current_query、runtime
stability	static、semi_static、session_dynamic、turn_dynamic、ephemeral
reuse_scope	global、agent_type、session、turn、request
cache_key	用于追踪稳定片段的确定性 hash
must_preserve	压缩或重排时必须保留的信息
source_pointer	可恢复压缩时指向原始内容的位置

5. Segment 稳定性分类

稳定性	示例	复用范围	推荐处理
static	system prompt、agent role、固定 tool schemas	跨请求/跨用户	放在最前面，固定格式
semi_static	repo summary、task plan、long-term memory summary	跨 session 或长周期	版本化，生成 cache key
session_dynamic	conversation history、short-term memory、之前 tool call 摘要	session 内复用	结构化、可分块
turn_dynamic	current query、current observation、当前 tool result	当前轮	放在后部，必要时压缩
ephemeral	timestamp、session id、retry count、random id	单次请求	后置或移出 prompt

基本原则：

- 越稳定、越可能复用的内容越靠前；
- 越动态、越一次性的内容越靠后；
- 能变成 metadata 的内容不要污染 prompt prefix。

6. 编译器 Pass 设计

Agent Context Compiler 可以设计成多个 pass，类似传统编译器。

6.1 Pass 1: Segment Parsing

目标：

- 把原始 Agent 状态解析成结构化 context segments。

输入示例：

```
system prompt
tools list
chat history
tool results
current query
runtime metadata
```

输出示例：

```
system segment
tool_schema segment
history segments
observation segments
current_query segment
runtime segment
```

这一层可以先靠规则实现，后续再适配 LangChain / AutoGen / SWE-Agent 的结构化对象。

6.2 Pass 2: Stability Analysis

目标：

判断每个 `segment` 是否稳定、是否值得放在 `prefix`、是否会污染 `cache`。

判断问题：

1. 这段是否跨请求相同？
2. 这段是否跨 turn 相同？
3. 这段是否只在当前 turn 有用？
4. 这段是否包含高熵字段？
5. 这段是否应该从 prompt 中移到 metadata？
6. 这段是否能生成稳定 cache key？

输出：

```
stability label
reuse_scope
cache_key
pollution risk
```

6.3 Pass 3: Canonicalization

目标：

让语义相同的上下文在文本层面也尽可能一致。

策略：

1. tool schema 使用 deterministic JSON serialization。
2. JSON key 固定排序。
3. tool list 固定排序，例如按 name、namespace、version。
4. 删除无意义随机空格和不稳定模板。
5. 固定 history 模板。
6. 固定 system/tool/memory 的 section header。

示例：

bad:

```
{"description": "...", "name": "search", "parameters": {...}}
```

better:

```
{"description":"...","name":"search","parameters":{"..."}}
```

6.4 Pass 4: Layout Planning

目标：

重新排列 segment，使稳定 prefix 最大化，动态污染最小化。

推荐布局：

[SYSTEM]
[TOOL_SCHEMAS]
[STABLE_MEMORY]
[SESSION_SUMMARY]
[RECENT_HISTORY]
[COMPRESSED_OBSERVATIONS]
[CURRENT_QUERY]
[RUNTIME_METADATA]
Assistant:

不推荐布局：

[TIMESTAMP]
[SESSION_ID]
[CURRENT_QUERY]
[OBSERVATION]
[TOOL_SCHEMAS]
[SYSTEM]
Assistant:

核心原则：

dynamic fields last
stable tools first
observations compressed and recoverable
history summarized or segmented

6.5 Pass 5: Tool-Aware Observation Compression

目标：

减少 observation token，同时保留任务成功所需的信息。

这不是普通摘要，而是根据工具类型做保真压缩。

Function calling

必须保留：

tool name
argument names
argument types
required fields
enum values
execution status

Coding agent

必须保留：

file path
symbol name
line number
error message
stack trace root cause
test failure
patch-relevant snippet

Retrieval / research agent

必须保留：

evidence sentence
source title
url/doc id
claim relation
conflicting evidence

Web agent

必须保留：

current goal
page state
visible interactive elements
last action
error feedback

6.6 Pass 6: Recoverable Reference Generation

目标：

压缩不是不可逆删除，而是 `summary + pointer`。

示例：

```
[OBSERVATION: pytest_result]
Tests failed.
Key error: AssertionError in tests/test_parser.py::test_nested_call
Last 20 stderr lines retained.
Full output recoverable at obs://run_42/pytest_stderr
```

好处：

1. 降低当前 prompt 成本。
2. 保留恢复完整 observation 的能力。
3. 避免一次压缩错误永久损害任务。
4. 更适合 long-horizon agent。

6.7 Pass 7: Quality Guard

目标：

在输出 `compiled prompt` 前检查关键字段是否被误删。

检查规则：

1. tool schema 是否仍包含 required fields。
2. JSON schema 是否仍合法。
3. coding observation 是否仍包含 file path、line number、error message。
4. retrieval observation 是否仍包含 evidence 和 source。
5. runtime metadata 是否没有污染 prefix。
6. 压缩比例是否超过任务允许阈值。

7. Cacheability 指标体系

7.1 Prefix Stability Ratio

$$\text{PSR} = \text{stable_prefix_tokens} / \text{total_input_tokens}$$

含义：

输入里有多少 token 位于稳定 prefix 中。

7.2 Dynamic Pollution Ratio

$$\text{DPR} = \text{dynamic_tokens_before_stable_boundary} / \text{total_input_tokens}$$

含义：

有多少动态 token 污染了可缓存前缀。

7.3 Cache Reuse Opportunity

$$\text{CRO}(i, j) = \text{longest_common_prefix}(\text{prompt}_i, \text{prompt}_j) / \min(\text{len}(\text{prompt}_i), \text{len}(\text{prompt}_j))$$

含义：

两个请求之间理论上有多少 prefix 可以复用。

7.4 Canonicalization Gain

$$\text{CG} = \text{prefix_overlap_after_compiler} - \text{prefix_overlap_before_compiler}$$

含义：

编译器让可复用 prefix 增加了多少。

7.5 Quality-Preserved Cost Reduction

QPCR = cost_reduction under quality_drop <= epsilon

示例：

在 tool accuracy 下降不超过 1% 的前提下，TTFT 降低多少？

在 task success rate 下降不超过 2% 的前提下，input token cost 降低多少？

8. 质量约束指标

不同任务需要不同的质量约束。

8.1 BFCL / Function Calling

质量指标：

1. JSON validity。
2. tool name accuracy。
3. argument key accuracy。
4. argument value accuracy。
5. execution success proxy。

8.2 SWE-Bench Lite / Coding Agent

质量指标：

1. patch applies。
2. test pass rate。
3. file localization accuracy。
4. edit localization accuracy。
5. tool call correctness。
6. error diagnosis preservation。

8.3 Retrieval / HoVer / Deep Research

质量指标：

1. answer correctness。

2. evidence recall。
3. citation accuracy。
4. fact verification accuracy。
5. contradiction preservation。

8.4 Web Agent / VisualWebArena

质量指标：

1. task success。
2. action accuracy。
3. loop rate。
4. page state consistency。

由于 PANDO 已经覆盖了 cache-aware prompting + skill distillation 的 Web Agent 场景，Web Agent 更适合作为补充实验，不建议作为主战场。

9. Baseline 设计

最小闭环 baseline：

1. Original prompt。
2. Stable tool ordering。
3. Dynamic fields last。
4. Truncation baseline。
5. Generic summarization baseline。
6. Agent Context Compiler without compression。
7. Agent Context Compiler with recoverable observation compression。

Serving baseline：

1. vLLM prefix cache off。
2. vLLM prefix cache on。
3. SGLang radix cache。
4. 如果后续需要，再加 session-sticky routing / prefix-hash routing。

注意：

routing 不应成为主贡献，否则会撞 KVFlow/PBKV/Preble。

10. 最小可行研究闭环

10.1 第一阶段：Synthetic Controlled Workload

目标：

验证 `compiler` 是否真的改善 `cacheability` 指标。

任务：

1. 构造 `original_bad_layout`。
2. 构造 `stable_tool_order`。
3. 构造 `dynamic_fields_last`。
4. 构造 `context_compiler_no_compression`。
5. 构造 `context_compiler_with_observation_compression`。
6. 构造 `truncation_baseline`。

验收：

1. Compiler 的 PSR 显著高于 original。
2. Compiler 的 DPR 显著低于 original。
3. Compiler 的 CRO 高于 original。
4. Compiler 不只是减少 token，还改善前缀结构。

10.2 第二阶段：Serving Latency

目标：

验证 `cacheability` 指标提升是否能转化为 TTFT/JCT 收益。

实验：

1. vLLM prefix cache off。
2. vLLM prefix cache on。
3. SGLang。

关键观察：

如果 `prefix cache on` 时 `compiler` 收益更明显，说明它确实改善了 `cache` 可利用性。

10.3 第三阶段：真实任务质量闭环

优先任务：

BFCL / function calling mini set

原因：

1. 质量指标清楚。
2. 工具 schema 是核心上下文。
3. JSON validity 和 tool accuracy 容易自动评估。
4. 比 SWE-Bench 更适合作为第一真实闭环。

下一步任务：

SWE-Bench Lite / coding agent traces

原因：

1. observation 更长。
2. 更能体现 recoverable compression 的价值。
3. 更有论文说服力。

11. 与已有工作的区别

11.1 与 KVFlow / PBKV

KVFlow/PBKV 做：

根据 workflow 或预测结果管理已经生成的 KV cache。

Agent Context Compiler 做：

在 KV cache 生成之前，优化 Agent 应该生成什么样的上下文。

11.2 与 PANDO

PANDO 做：

面向 multimodal web agent 的 online skill distillation 和 cache-aware prompting。

Agent Context Compiler 做：

面向通用 tool-using agent 的结构化上下文编译。

避免重合的方法：

1. 不主打 WebArena。
2. 不做 skill distillation。
3. 主打多类工具任务和通用 segment compiler。

11.3 与 ACE

ACE 做：

通过执行反馈动态改进 Agent 上下文，提高任务成功率。

Agent Context Compiler 做：

根据 cacheability 和质量约束编译上下文，提高 serving efficiency。

11.4 与 Prompt Compression

Prompt compression 做：

减少 token。

Agent Context Compiler 做：

减少不必要 token，同时提高 prefix/cache 可利用性，并保证工具任务质量。

11.5 与 ToolCaching / TVCache

ToolCaching / TVCache 做：

缓存工具调用结果或工具状态。

Agent Context Compiler 做：

不复用工具结果，只压缩、引用和重排 `observation`。

12. 论文贡献表述

可以写成：

We propose Agent Context Compiler, a framework that transforms raw tool-using agent contexts into

中文：

我们提出 **Agent Context Compiler**，一个在任务质量约束下，将原始工具调用智能体上下文转换为缓存友好布局的

核心贡献：

1. 提出 agent context cacheability 的指标体系。
2. 设计通用上下文编译器，包括 segment parsing、stability analysis、canonicalization、layout planning、tool-aware recoverable compression。
3. 在多类工具任务上证明，compiler 能在保持任务质量的同时降低 input token cost、TTFT/JCT，并提高 prompt/prefix cache 可利用性。
4. 系统分析 compiler 有效和失效的边界条件。

13. 当前 MVP 状态

当前仓库已经有最小骨架：

```
projects/agent_infer_bench/agent_bench/optimizations/context_compiler.py
projects/agent_infer_bench/agent_bench/analysis/cacheability_metrics.py
projects/agent_infer_bench/scripts/analysis/analyze_context_compiler.py
projects/agent_infer_bench/configs/week07_context_compiler.yaml
```

当前已支持：

1. ContextSegment 抽象。
2. deterministic JSON serialization。
3. static / dynamic / ephemeral layout planning。

4. observation recoverable compression。
5. PSR / DPR / CRO 指标。
6. synthetic controlled ablation。

下一步：

1. 接 vLLM prefix cache on/off。
2. 接 SGLang radix cache。
3. 加 BFCL mini quality evaluation。
4. 加 SWE-Bench Lite/coding trace observation compression。
5. 把 mock latency 替换为真实 serving latency。

14. Go / No-Go 标准

继续推进的 Go 条件：

1. Compiler 稳定提高 PSR/CRO，降低 DPR。
2. 在真实 prefix cache backend 上，cacheability 提升能转化为 TTFT/JCT 改善。
3. Compiler 优于 stable tool ordering 和 truncation baseline。
4. 在 BFCL mini 上，JSON validity 和 tool accuracy 基本不下降。
5. 在 coding trace 上，recoverable compression 比普通 truncation 更稳。

停止或降级的 No-Go 条件：

1. Compiler 收益和简单 truncation 一样。
2. 质量明显下降。
3. 真实 backend 上 latency 没有任何改善。
4. 指标只能在 synthetic workload 上成立。
5. 方法无法和 PANDO / ACE / prompt compression 拉开边界。

15. 最重要的研究边界

不要做：

1. KV eviction。
2. KV prefetch。
3. future agent invocation prediction。
4. tool result cache。
5. cross-agent KV tensor sharing。

6. 只服务 Web Agent 的 skill distillation。

要做：

1. 通用 tool-using agent context segmentation。
2. cacheability-aware layout planning。
3. deterministic tool schema canonicalization。
4. tool-aware recoverable observation compression。
5. quality-preserved cost reduction。
6. 多类工具任务上的端到端验证。